Introduction to generalized additive models

day 1 – part 2

Jürgen Claesen

Amsterdam UMC, VU Amsterdam, The Netherlands

March 23, 2026

Overview

I. Introduction

II. Splines – part I: One-dimensional smoothers

III. Splines – part II: Multi-dimensional smoothers

IV. Hypothesis testing

Introduction

Introduction

Recall an additive model with one covariate (x):

$$y_i = \alpha + f(x_i) + \varepsilon_i$$

with

$$f(x) = \sum_{j=1}^{k} b_j(x)\beta_j$$

where b_j is the jth basis function which is completely known

Introduction

To find estimates for $\boldsymbol{\beta}$, we use penalized regression:

$$\underset{\beta}{\operatorname{argmin}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda \boldsymbol{\beta}^T \mathbf{S} \boldsymbol{\beta}$$

where λ is the smoothing parameter and $\boldsymbol{\beta}^T \mathbf{S} \boldsymbol{\beta}$ is a penalty.

Additive models

Mixed effects models

We can reparametrize additive models as follows:

$$\begin{aligned} y_i &= \beta_0 + f(x_i) + \varepsilon_i \\ &= \beta_0 + \beta_1 x_i + f^*(x_i) + \varepsilon_i \end{aligned}$$

with

$$f^*(x_i) = \sum_{j=1}^{K} b_j(x_i)\beta_j^*$$

Additive models

Mixed effects models

Defining

$$\boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} \text{ and } \mathbf{u} = \begin{bmatrix} \beta_1^* \\ \vdots \\ \beta_K^* \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \text{ and } \mathbf{Z} = \begin{bmatrix} b_1(x_1) & \dots & b_K(x_1) \\ \vdots & \ddots & \vdots \\ b_1(x_n) & \dots & b_K(x_n) \end{bmatrix}$$

the reparametrized additive model has following form:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}\mathbf{u} + \boldsymbol{\varepsilon}$$

$\rightarrow$ standard form of a linear mixed model

Additive models

Mixed effects models

The mixed effect model:

$$\begin{aligned} \mathbf{y} &= \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}\mathbf{u} + \boldsymbol{\varepsilon} \\ \mathbf{u} &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}\sigma^2/\lambda) \\ \boldsymbol{\varepsilon} &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}\sigma^2) \end{aligned}$$

can be estimated by minimizing the penalised regression criterion:

$$\frac{1}{\sigma^2}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta} - \mathbf{Z}\mathbf{u}\|^2 + \frac{\lambda^2}{\sigma^2}\|\mathbf{u}\|^2$$

or as is common for mixed effects models by ML or REML.

Additive models

Mixed effects models – Practical

```
> library(nlme)
> data(lidar)
>
> nbKnots <- 12
> sD <- seq(min(lidar$range), max(lidar$range),
+ length=200) ## prediction data
> sj2 <- seq(min(lidar$range),max(lidar$range),
+ length=nbKnots) ## generate knots
> X0 <- tf.X(lidar$range,sj2) ## X in original parametr.
> Xp.orig <- tf.X(sD,sj2) ## Xpred in original parametr.
> D <- rbind(0,0,diff(diag(nbKnots),difference=2))#sqrt per
> diag(D) <- 1
> X <- t(backsolve(t(D),t(X0))) ## re-parameterized X
> Z <- X[,-c(1,2)]; X <- X[,1:2] ## mixed model matrices
> Xp1 <- t(backsolve(t(D),t(Xp.orig))) ## re-parametrized X
```

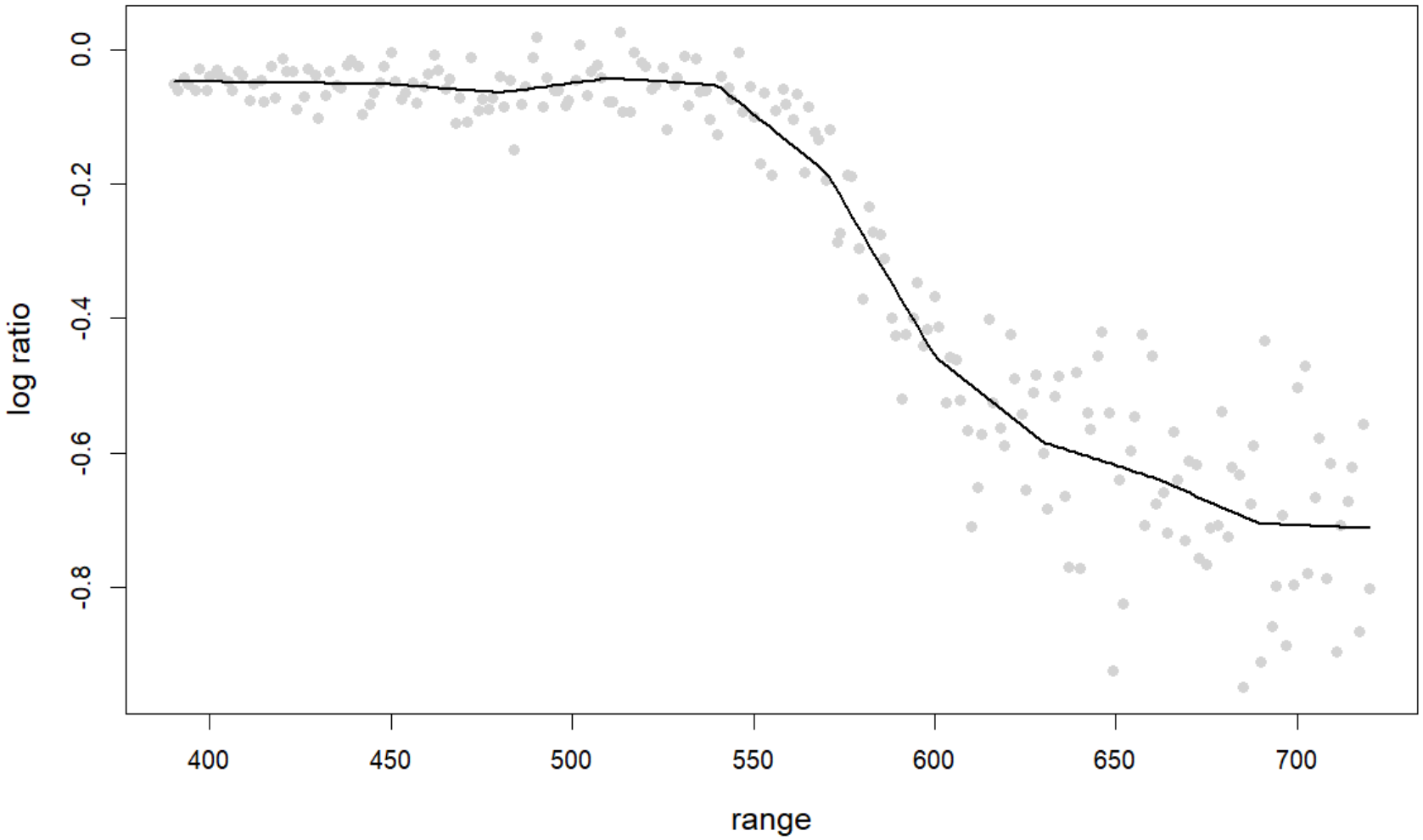
Additive models

Mixed effects models – Practical

```
> g <- factor(rep(1,nrow(X))) ## dummy factor
> m <- lme(logratio ~ X - 1,
+ random=list(g = pdIdent(~ Z-1)), data=lidar) #fit model
>
> plot(x=lidar$range,lidar$logratio,
+ pch=16, cex.lab=1.25, col="lightgray",
+ xlab="range", ylab="log ratio")
> lines(sD,Xp1 %*% as.numeric(coef(m)), lwd=2, col=1)
```

Additive models

Mixed effects models – Practical



Additive models

Mixed effects models – Practical

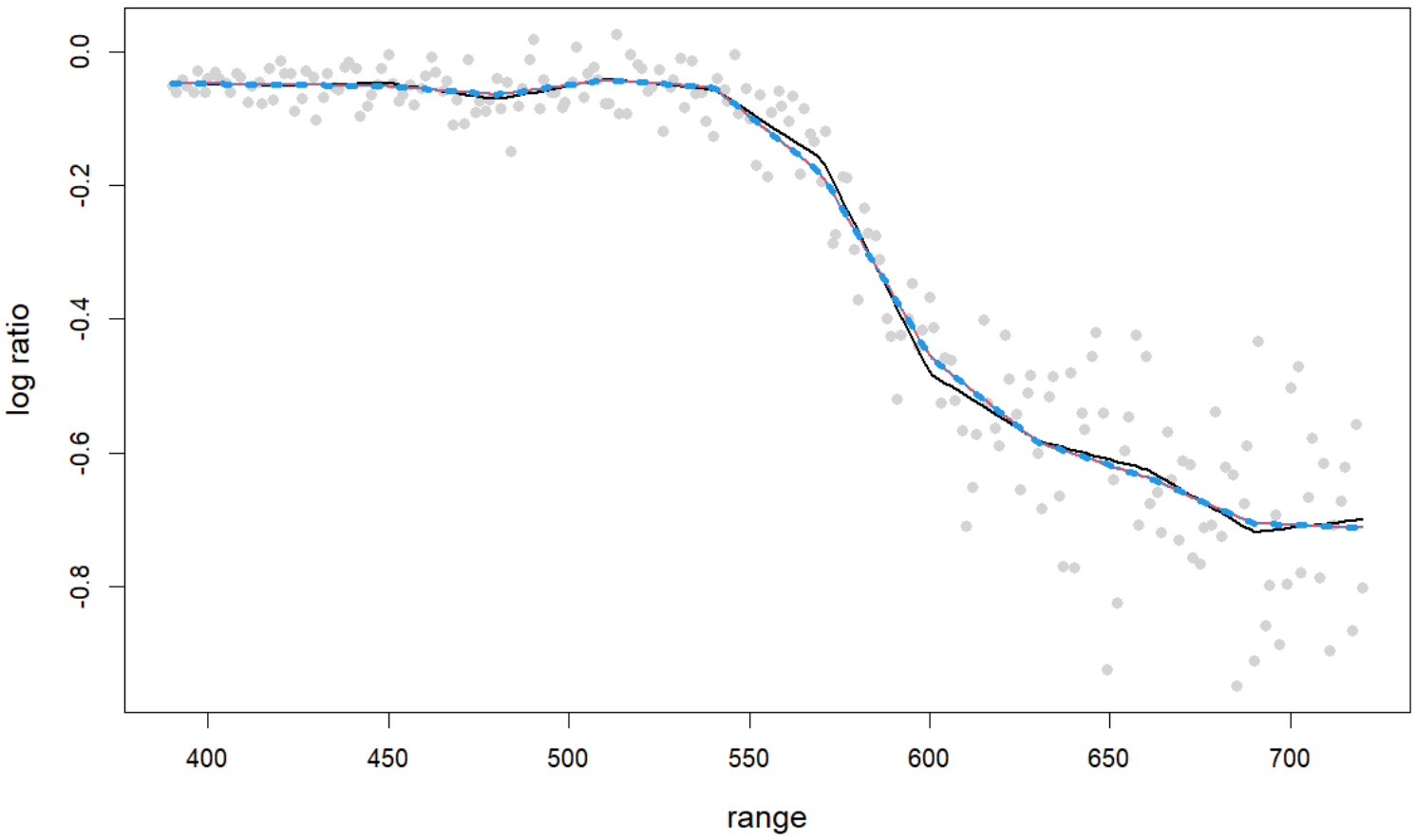


Figure: Fitted model estimated without penalization (black), with penalization (red) and with REML (dotted line, blue)

Exercise 6

Splines – part I: One-dimensional smoothers

Introduction

Assume a generalized additive model with one covariate (x):

$$g(\mu_i) = \alpha + f(x_i)$$

with

$$f(x) = \sum_{j=1}^{k} b_j(x)\beta_j$$

where b_j is the jth basis function which is completely known

Introduction

For example the tent function basis:

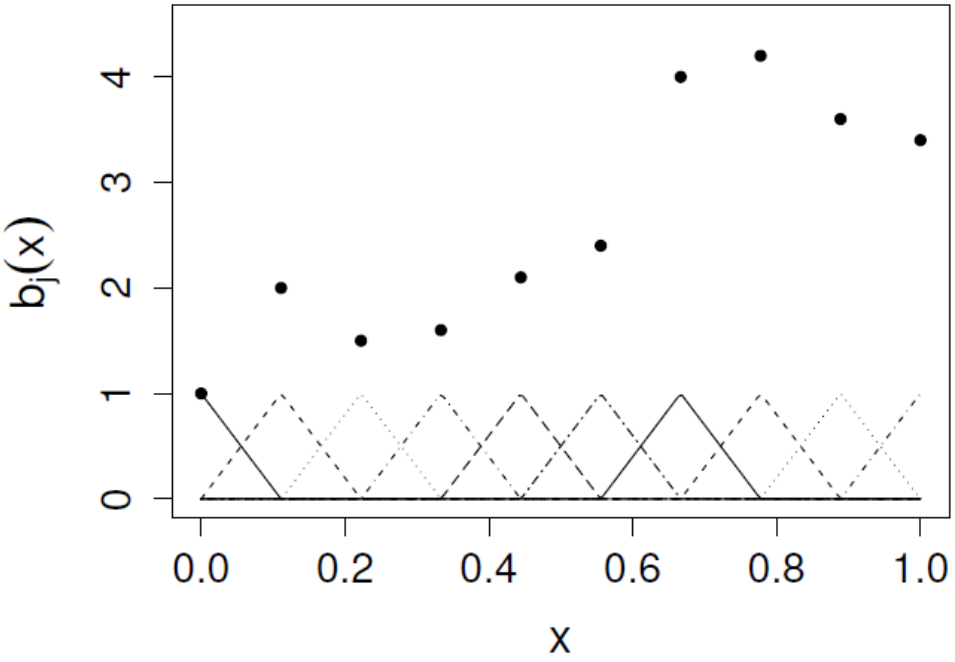
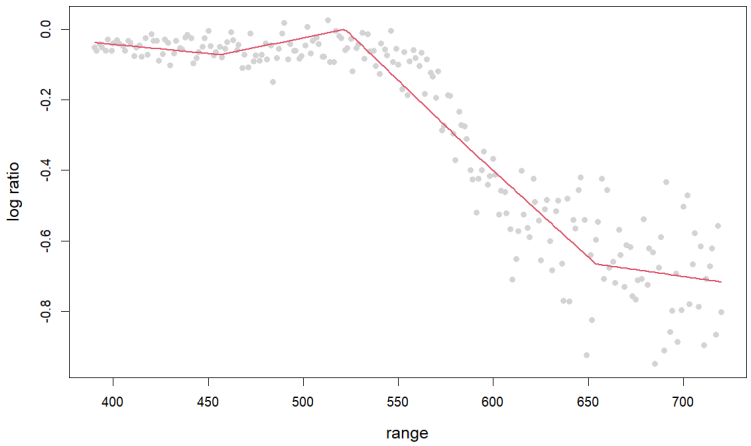
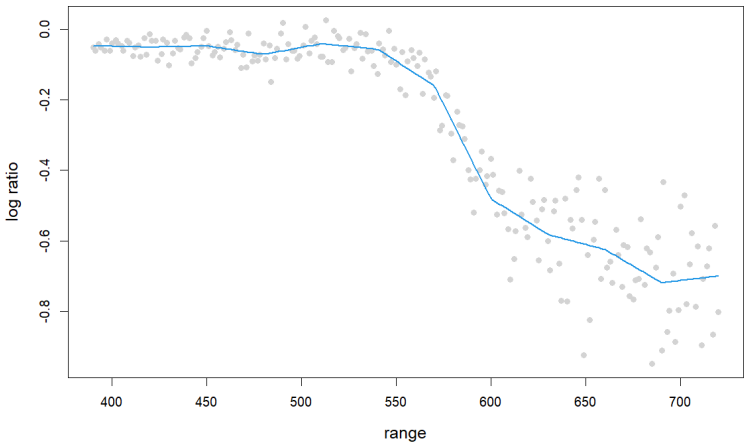


Figure: Tent function basis for interpolating the data (black dots)[1].

[1] Taken from Wood, 2017

Piecewise linear basis ⇒ non-smooth scatterplot smoothers

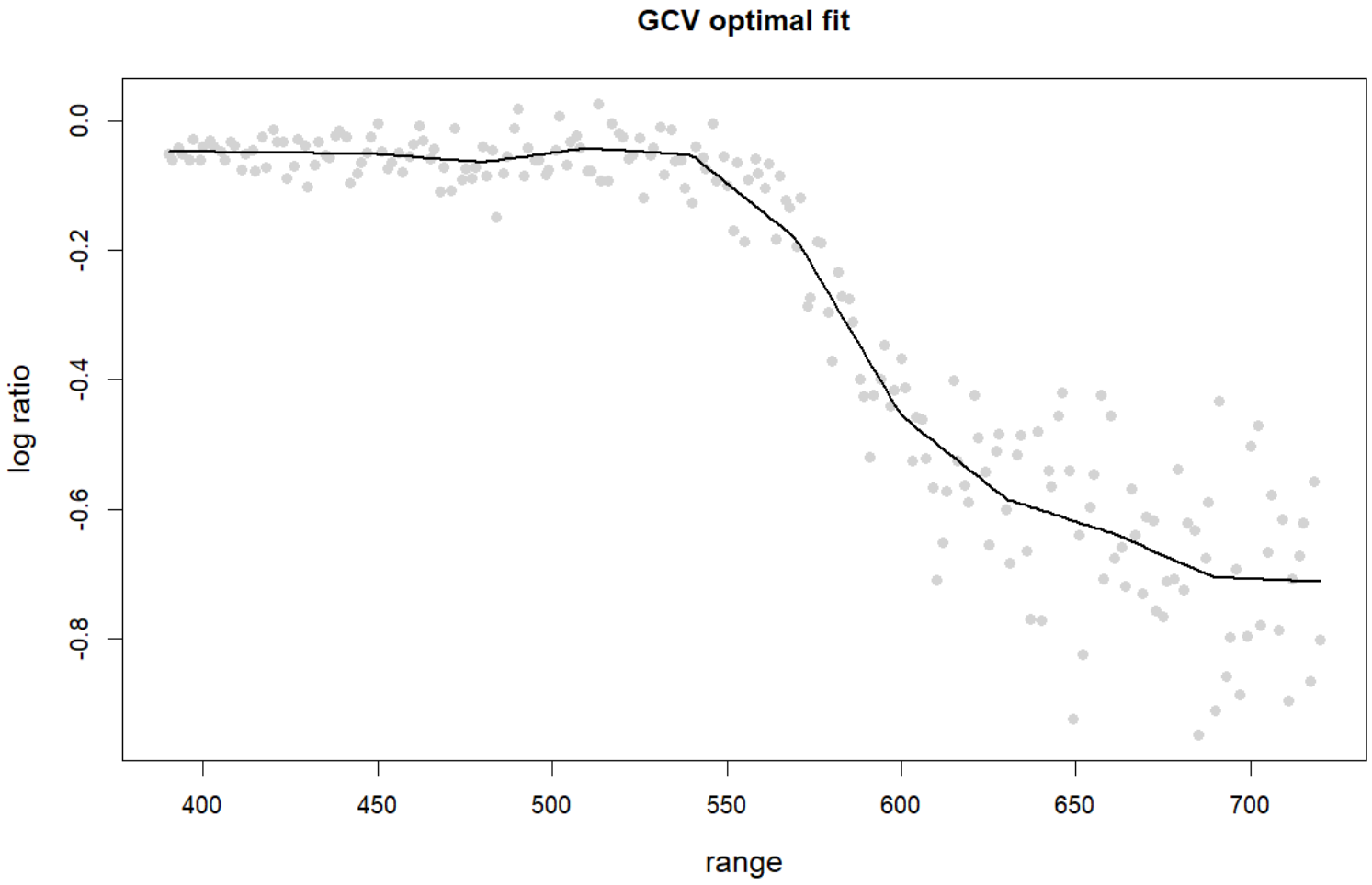




Introduction

Alternatives to piecewise linear basis

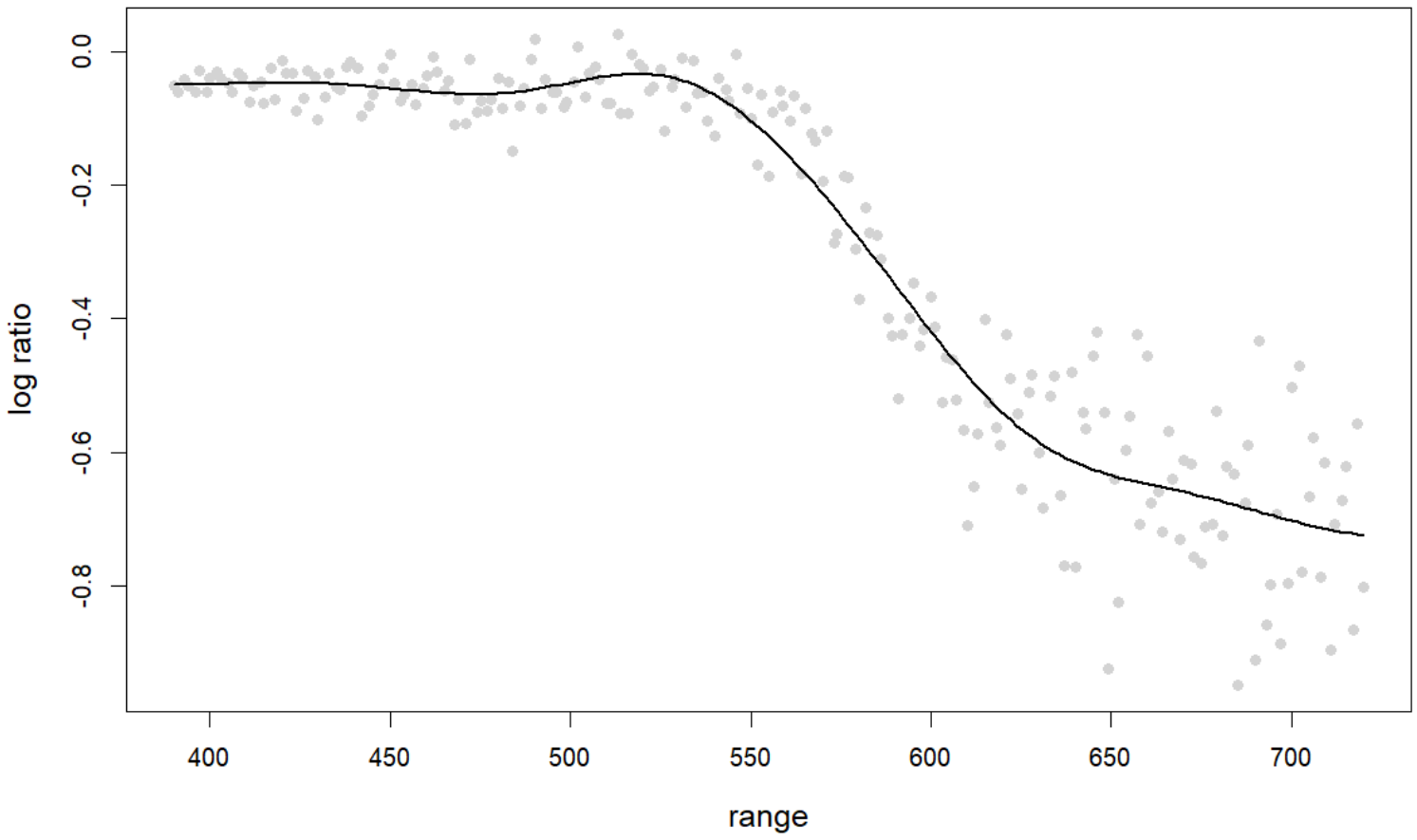
Even after penalization of a *reasonably large* number of knots, we have non-smooth smoothers



Introduction

Alternatives to piecewise linear basis

Increasing the order of the basis function (quadratic or cubic basis functions) leads to smooth smoothers.



Splines

Introduction

Generalized additive model with one covariate (x):

$$\eta(\mu_i) = \alpha + f(x_i)$$

with

$$f(x) = \sum_{j=1}^{k} \beta_j b_j(x)$$

where the basis functions b_j are piece-wise polynomials $\rightarrow$ refer to $f(x)$ as a *spline*

Smoothness is controlled by the number of knots $\rightarrow$ use penalized regression to determine level of smoothness

Splines

Penalized regression splines

Estimate $\hat{\beta}$:

$$\underset{\beta}{\operatorname{argmin}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 + \lambda \int f''(x)^2 dx$$

where λ is the smoothing parameter and

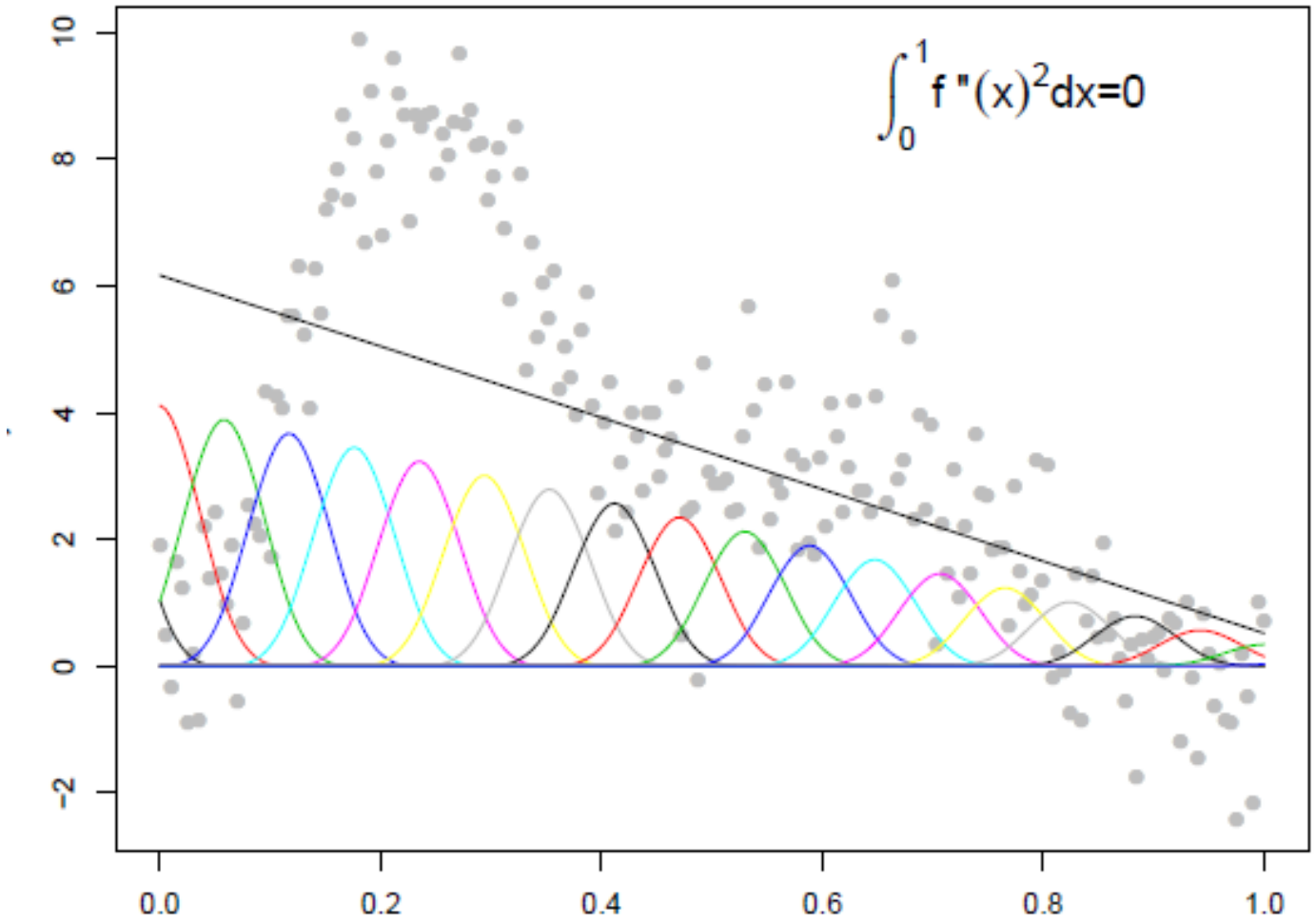
$$f''(x) = \sum_{j=1}^{p} \beta_j b_j''(x)$$

$$\int f''(x)^2 dx = \boldsymbol{\beta}^T \mathbf{S} \boldsymbol{\beta}$$

$$\text{where } S_{jk} = \int b_j''(x) b_k''(x) d(x)$$

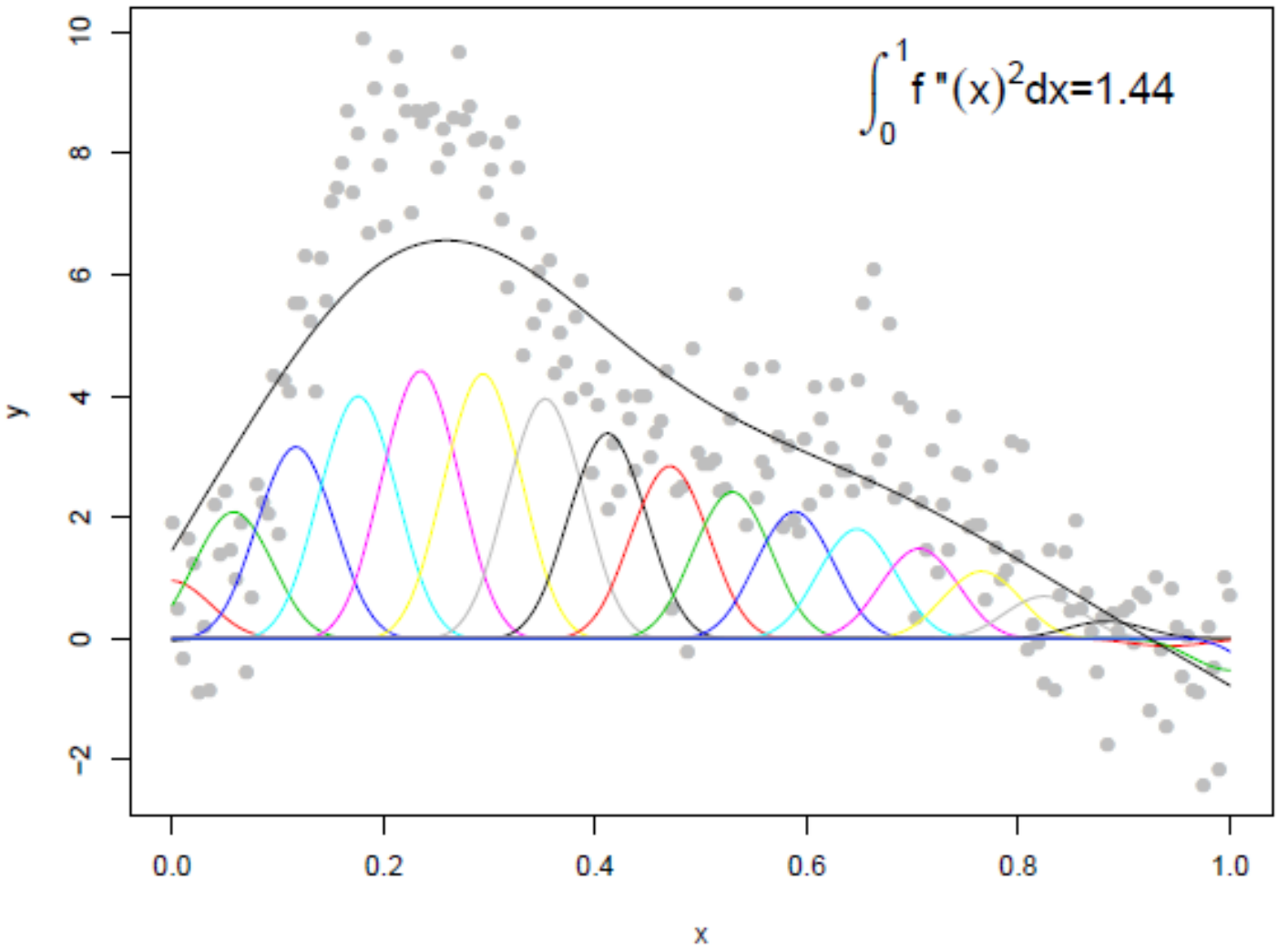
Splines

Penalized regression splines



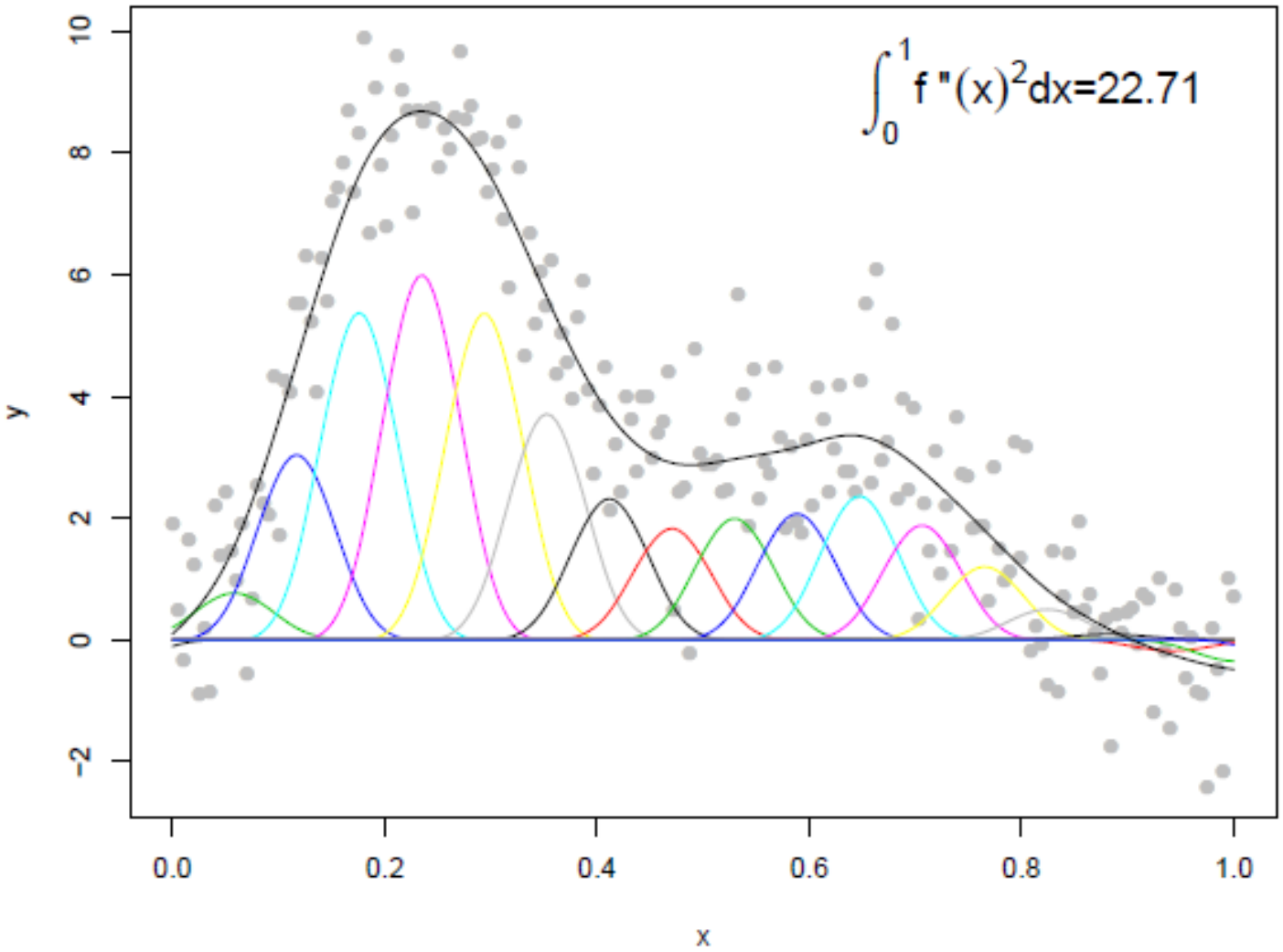
Splines

Penalized regression splines



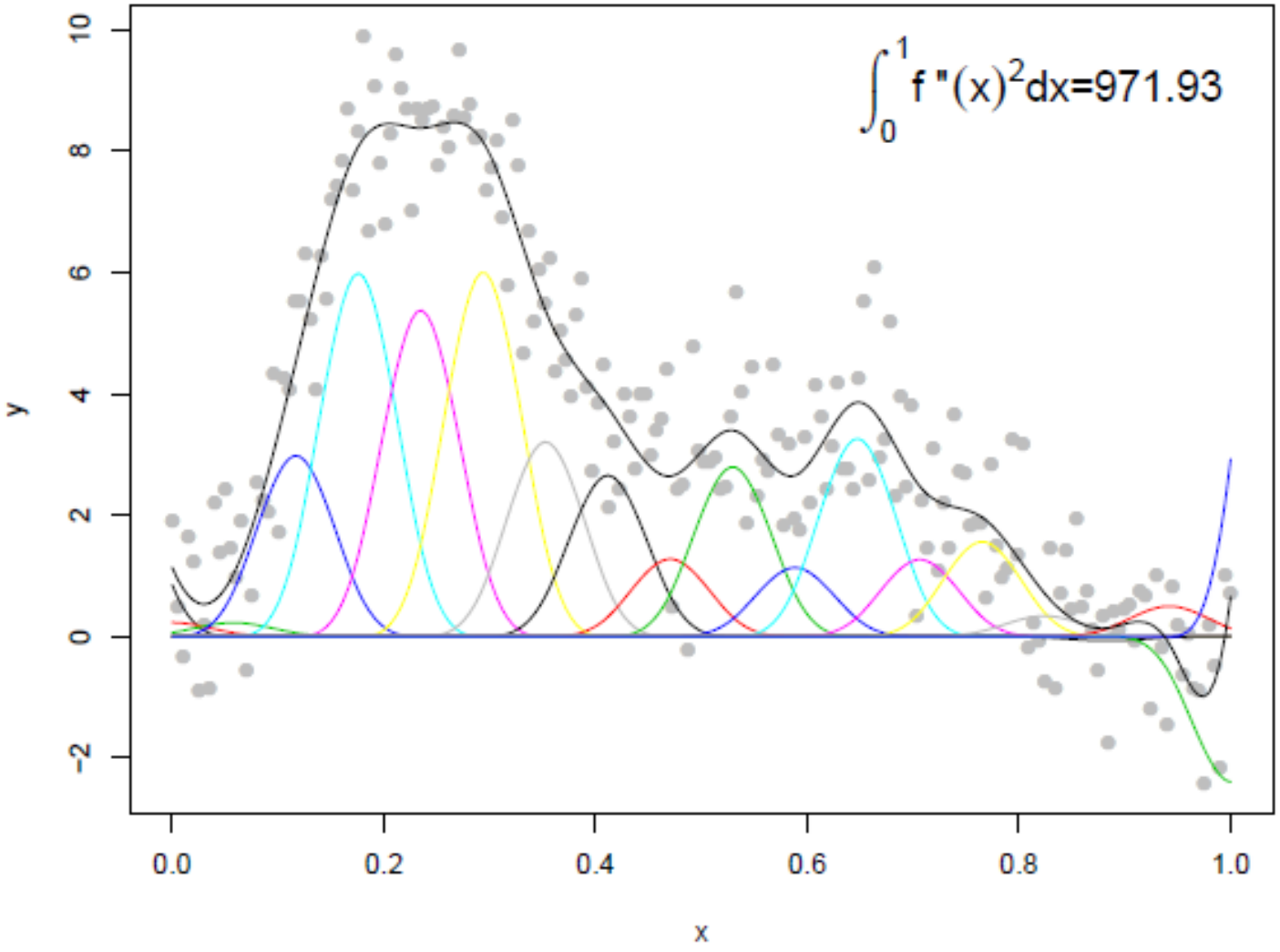
Splines

Penalized regression splines



Splines

Penalized regression splines



Choice of the number of knots is less important because of the penalized regression.

However, a reasonable number of knots should be chosen:

- $k \approx n$
- $k \ll n$ (low-rank alternative)

Low-rank alternative is computationally less demanding and makes extension to complicate models more straightforward

Choice of the number of knots:

- number of knots determines how "flexible" $\hat{f}(x)$ is
- impossible to find the optimal number $\rightarrow$ need to know the real underlying function
- in practice: the number of knots is the upper limit $\rightarrow$ the actual flexibility is controlled by the smoothing parameters
- part of model specification

Choice of the number of knots $\rightarrow$ can be informally checked:

- residual randomization test (implemented in *gam.check*())
- extract deviance residuals and smooth them with a higher number of knots (e.g. 2k) $\rightarrow$ very smooth and nearly zero (no pattern present)

Choosing λ:

- (generalized) cross-validation
- AIC or Mallow's Cp
- REML (mixed model approach)

Splines

One-dimensional smoothers

Several basis functions are useful for smoothing wrt a single predictor

Two commonly used splines:

- cubic regression splines
- P-splines

Splines

Cubic regression splines

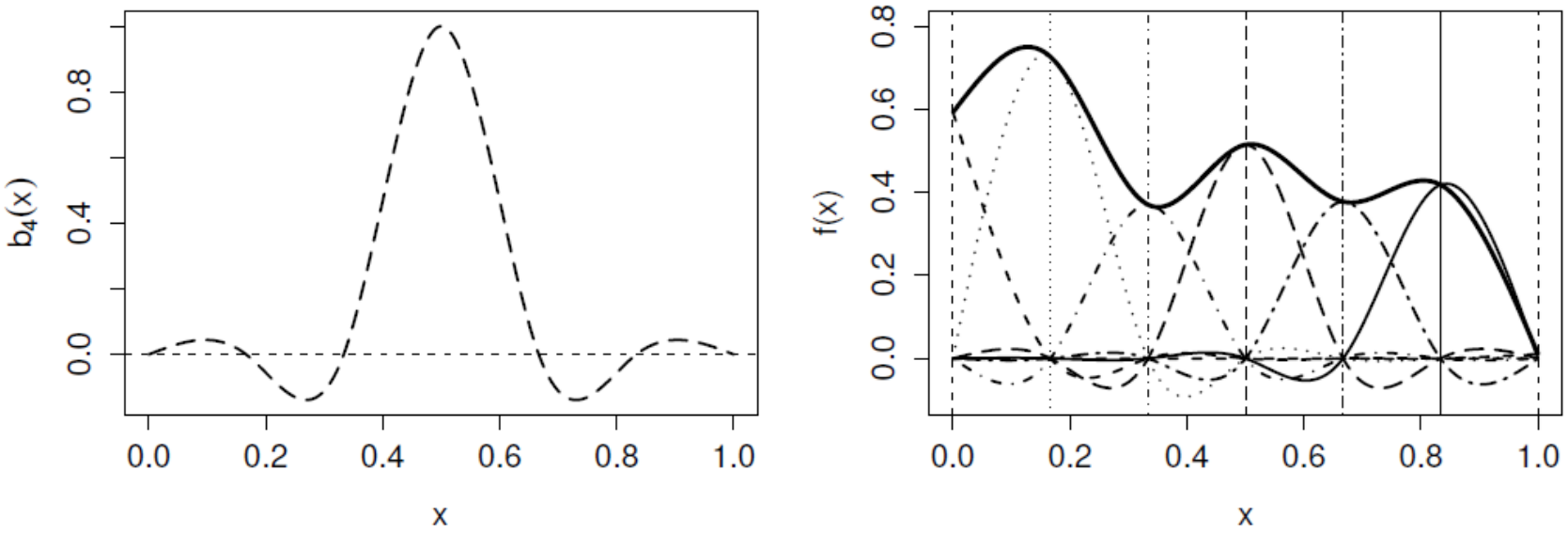


Figure: The left panel shows a basis function ($b_4(x)$) for a cubic regression spline. The right panel shows how such basis functions are combined to represent a smooth curve.[3]

[3] Taken from Wood, 2017

An $(m+1)^{th}$ order B-spline can be represented as:

$$f(x) = \sum_{i=1}^{k} B_i^m(x)\beta_i$$

with $k+m+2$ knots: $x_1^* < x_2^* < \cdots < x_{k+m+2}^*$

$$B_i^m(x) = \frac{x - x_i^*}{x_{i+m+1} - x_i^*} B_i^{m-1}(x) + \frac{x_{i+m+2}^* - x}{x_{i+m+2}^* - x_{i+1}^*} B_{i+1}^{m-1}(x)$$

and

$$B_i^{-1}(x) = \begin{cases} 1 \text{ if } x_i^* \leq x \leq x_{i+1}^* \\ 0 \text{ otherwise} \end{cases}$$

Splines

B-splines

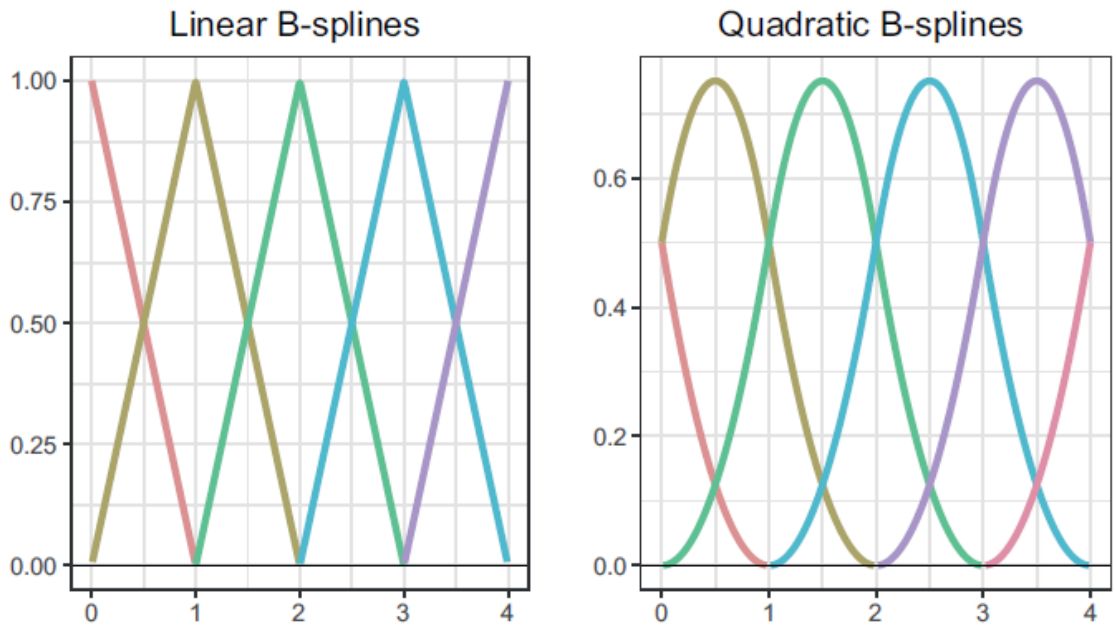


Figure: Linear (left) and quadratic (right) B-spline bases[4]

[4] Taken from Eilers and Marx, 2021

Splines

P-splines

P-splines:

- low rank smoothers with B-spline basis functions
- usually on evenly spaced knots
- difference penalty applied directly to β
- difference penalty can have different orders

For example, the difference penalty with $m = 1$:

$$\mathcal{P} = \sum_{i=1}^{k-1} (\beta_{i+1} - \beta_i)^2 = \boldsymbol{\beta}^T \mathbf{P}^T \mathbf{P} \boldsymbol{\beta}$$

or, the difference penalty with $m = 2$:

$$\mathcal{P} = \sum_{i=2}^{k-1} (\beta_{i-1} - 2\beta_i + \beta_{i+1})^2$$

Any order of the B-spline basis functions can be combined with any order of the difference penalty $\Rightarrow$ P-splines are very flexible

P-splines allow adaptive smoothing:

The amount of smoothing varies with the smoothing covariate.

For a difference penalty with $m = 2$:

$$\mathcal{P}_a = \sum_{i=2}^{k-1} \omega_i (\beta_{i-1} - 2\beta_i + \beta_{i+1})^2 = \boldsymbol{\beta}^T \mathbf{P}^T \text{diag}(\boldsymbol{\omega}) \mathbf{P} \boldsymbol{\beta}$$

The weights ω_i vary smoothly with i and hence with x.

Splines – part II: Multi-dimensional smoothers

Introduction

Multi-dimensional smoothers: consider smooths of more than one variable:

- isotropic smoothing
- tensor product smooth interactions
- ANOVA decomposition of multi-dimensional smoothers

Isotropic smoothing

Smoothers for multiple covariates at one:

$$y_i = f(\mathbf{x}_i) + \varepsilon_i$$

with $\mathbf{x}_i$ a d-vector and $f(\mathbf{x}_i)$ an isotropic smoother

They produce identical predictions of the response variable under any rotation or reflection of the covariates $\rightarrow$ modelling geographical data (coordinates), data on a sphere ...

Isotropic smoothing

Isotropic smoothers:

- thin plate splines
- thin plate regression splines
- Duchon splines
- ...

Tensor product smooth interactions

Isotropic smoothers use a flexible multidimensional "strip" to smooth $f(\mathbf{x}_i) \rightarrow$ requires scaling of the covariates relative to one another

Issue: How do you scale fundamentally different covariates? e.g. spatial coordinates and time?

Tensor product smooths: in contrast to isotopic smoothers they use a lattice of flexible "strips", each with a different stiffness in different directions

Tensor product smooth interactions

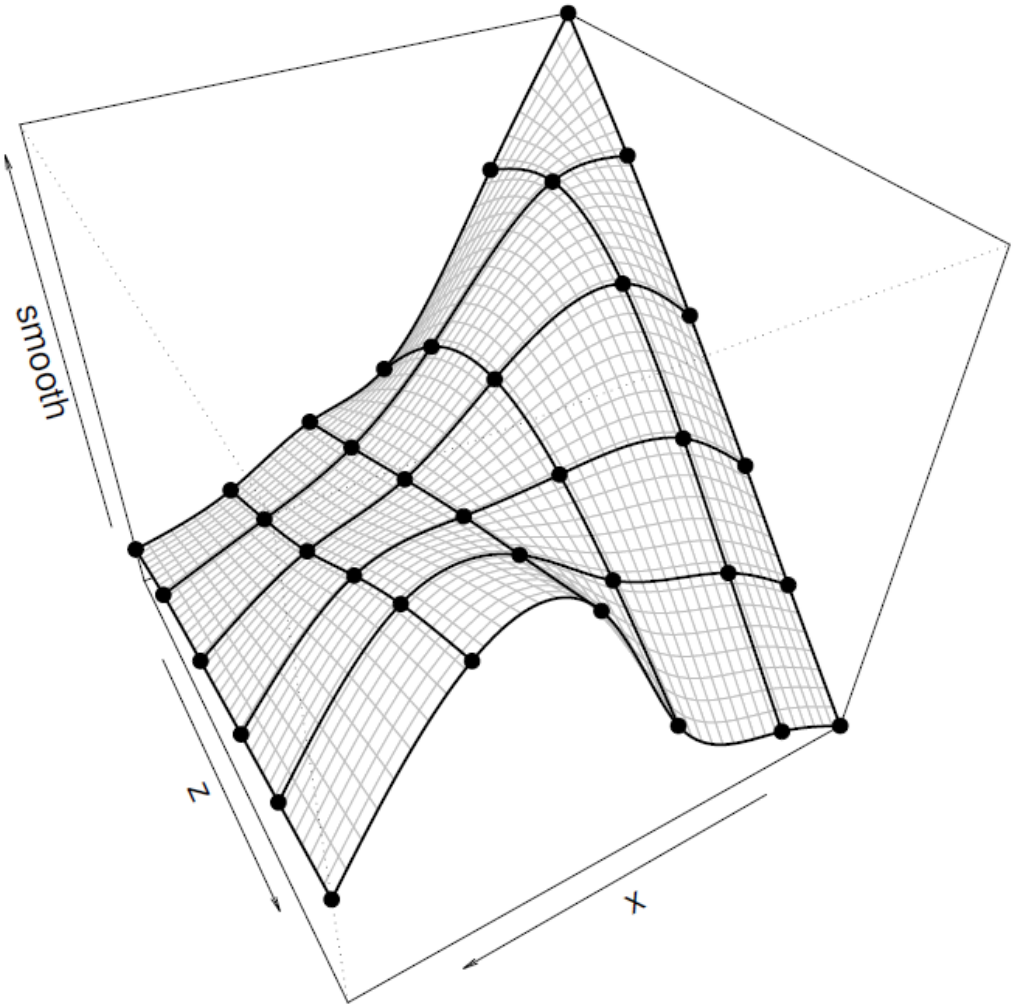


Figure: Illustration of a tensor product smooth of two variables x and z[5]

[5] Taken from Wood, 2017

Tensor product smooth interactions

Basic approach:

i. start from smooths of single covariates: any basis and quadratic penalty

ii. construct a tensor product to build up smooths of several variables

Resulting smooths are invariant to rescaling

Tensor product smooth interactions

ANOVA decomposition of smooths

Sometimes useful to "decompose" tensor product smooth interactions

For example:

$$f_1(x) + f_2(z) + f_3(x, z)$$

with $f_1(x)$, $f_2(z)$ the main effects and $f_3(x, z)$ the interaction effect

$f_3(x, z)$ should not include any information that is captured by $f_1(x)$ and $f_2(z)$

Hypothesis testing

Introduction

Hypothesis testing is an alternative for model selection, especially when we have reason to favour simpler models over more complicated ones

Hypothesis testing

<u>Parametric model effects</u>:

unpenalized parameters $\rightarrow$ traditional inference for linear regression models

<u>Smoothers</u>:

penalized parameters $\rightarrow$ shouldn't use traditional inference

approximate p-values for smooth terms:

H_0: $f(x) = 0$ instead of H_0: $\beta_j = 0$

Effective degrees of freedom

Approximate p-values are connected to EDF

EDF: degrees of freedom corrected for penalisation, helps with overfitting

High EDF-value $\rightarrow$ more wiggly, more flexible smoother.

EDF $\approx 1 \rightarrow$ straight line $\rightarrow$ no smoother needed

Confidence intervals

Penalised regression and mixed effect model implementation both allow to construct point-wise confidence intervals.

Mixed effects model are also linked to Bayesian model (penalty is the prior) $\Rrightarrow$ Confidence interval are actually credible intervals

Alternative to point-wise confidence interval are simultaneous confidence bands: C.I. for whole $\hat{f}(x)$

mgcv-package

R-package *mgcv*

Introduction

Main function in the R-package *mgcv* is *gam*()

- similar to the *glm*()-function
- can handle parametric model terms and smooth terms (*s*() and *te*())
- options for controlling automatic smoothing parameter estimation, or for directly controlling model smoothness

R-package *mgcv*

```
> library(mgcv)
> library(SemiPar)
> data(lidar)
> fitLidar <- gam(logratio~s(range, bs="cr", k=12),
+  family=gaussian, data=lidar)
```

R-package *mgcv*

This fits following additive model:

$$\text{log ratio}_i = \text{f(range)}_i + \varepsilon_i$$

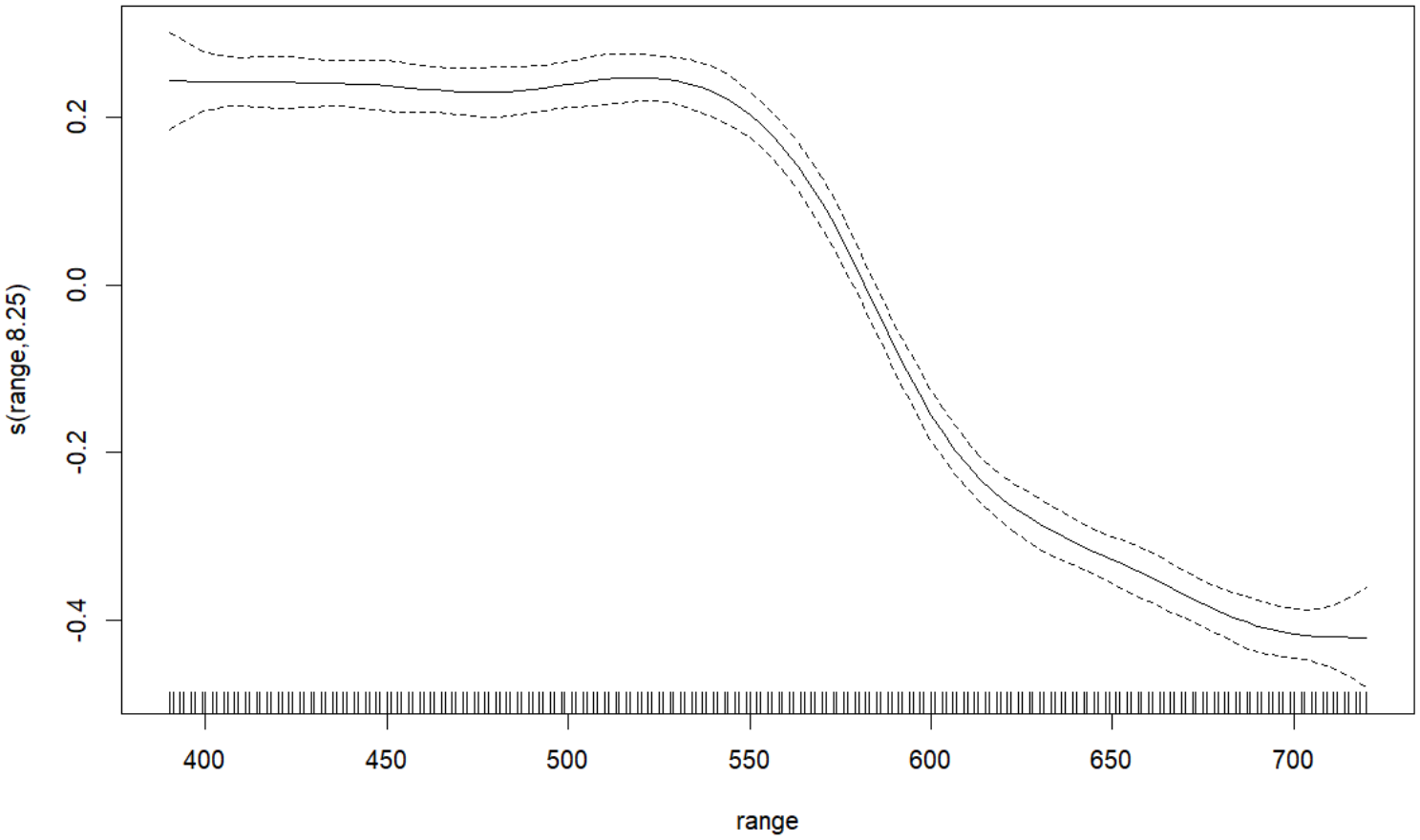
with:

- f(range)_i: a penalized cubic regression spline (`bs="cr"`) with 12 knots (`k=12`)
- normal distributed data (default)
- GCV is used to determine λ (default)

R-package *mgcv*

```
> fitLidar

Family: gaussian
Link function: identity

Formula:
logratio ~ s(range, bs = "cs", k = 12)

Estimated degrees of freedom:
8.25  total = 9.25

GCV score: 0.006554065
> plot(fitLidar)
```

R-package *mgcv*



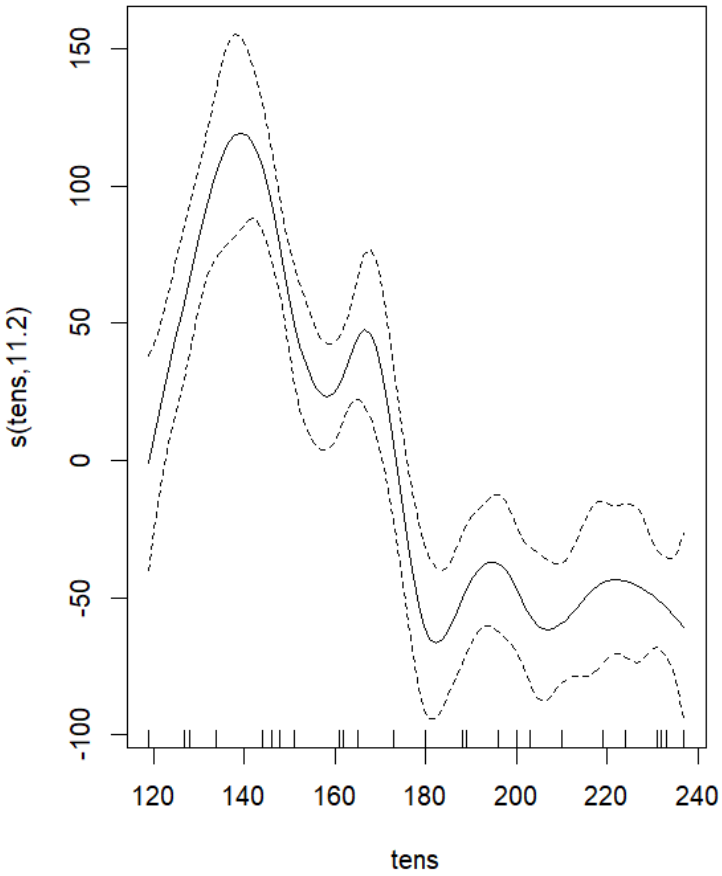
R-package *mgcv*

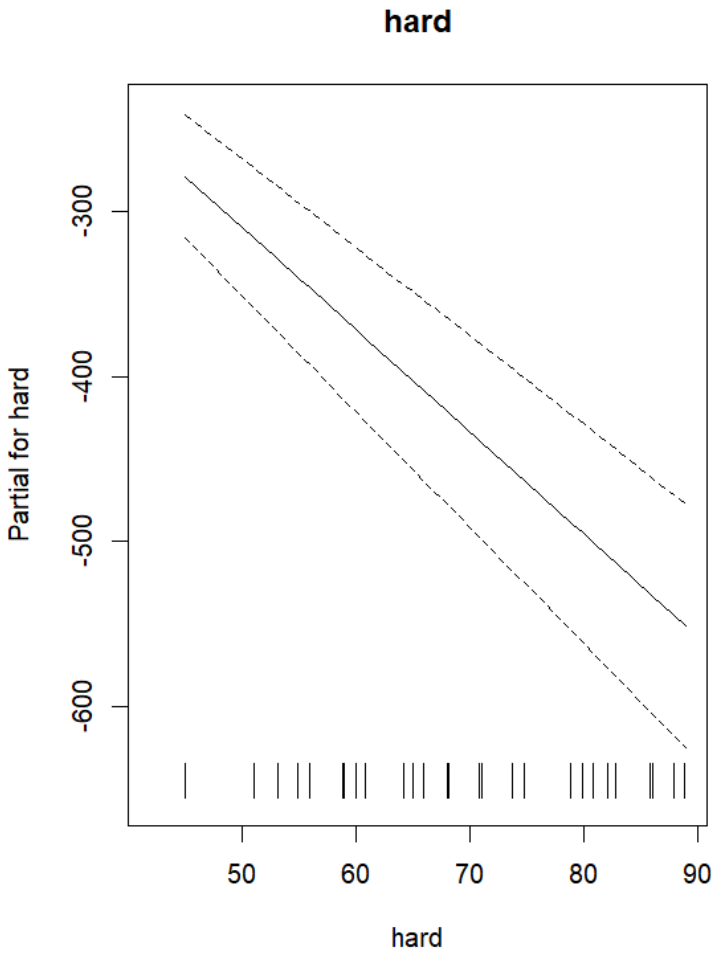
```
> library(MASS)
> data(Rubber)
> fitRubber1 <- gam(loss~hard+s(tens, bs="cr", k=15),
+  family=gaussian, data=Rubber)
> fitRubber1
> par(mfrow=c(1,2))
> plot(fitRubber1, all.terms=TRUE)
```

R-package *mgcv*

```
Family: gaussian
Link function: identity

Formula:
loss ~ hard + s(tens, bs = "cr", k = 15)

Estimated degrees of freedom:
11.2  total = 13.2

GCV score: 628.8437
```

R-package *mgcv*





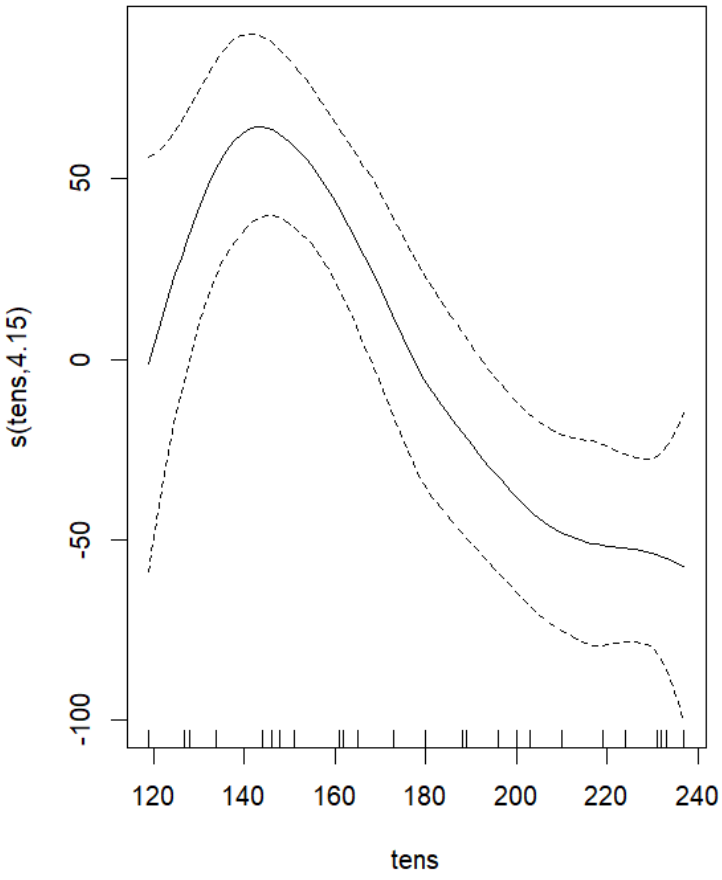
R-package *mgcv*

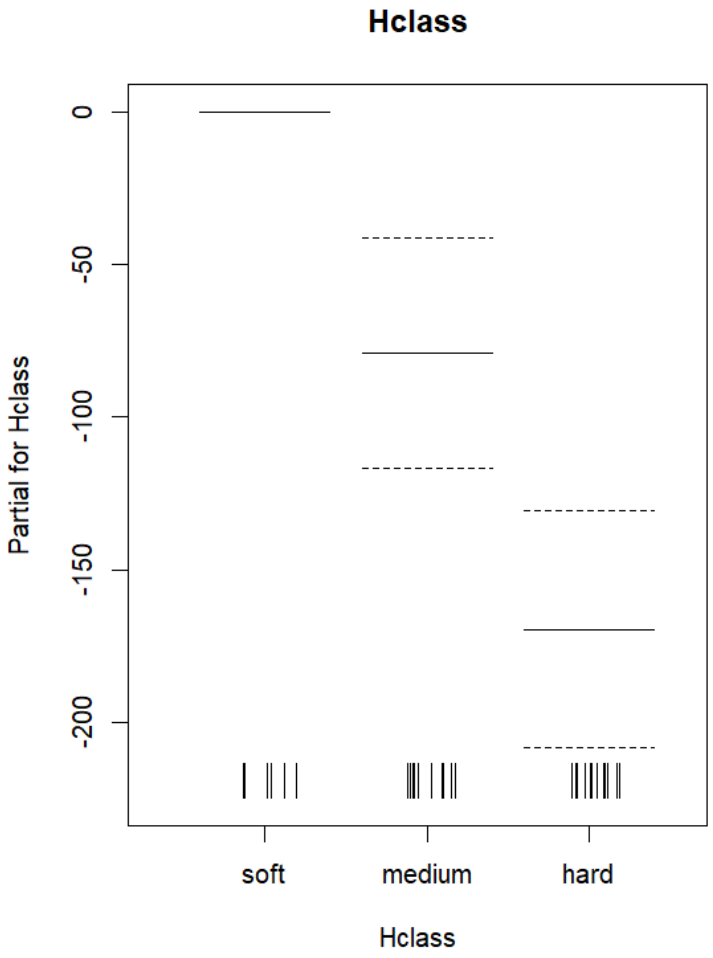
```
> Rubber$Hclass <- factor(floor(Rubber$hard/15)-3,
+ labels=c("soft","medium","hard"))
> fitRubber2 <- gam(loss~Hclass+s(tens, bs="cs", k=15),
+  family=gaussian, data=Rubber)
> fitRubber2
> par(mfrow=c(1,2))
> plot(fitRubber2, all.terms=TRUE)
```

R-package *mgcv*

```
Family: gaussian
Link function: identity

Formula:
loss ~ Hclass + s(tens, bs = "cs", k = 15)

Estimated degrees of freedom:
4.15  total = 7.15

GCV score: 1804.314
```

R-package *mgcv*





R-package *mgcv*

```
> anova(fitRubber2)
Family: gaussian
Link function: identity

Formula:
loss ~ Hclass + s(tens, bs = "cs", k = 15)

Parametric Terms:
       df     F  p-value
Hclass  2 38.82 4.48e-08

Approximate significance of smooth terms:
           edf Ref.df     F  p-value
s(tens)  4.151 14.000 3.267 1.62e-05
```

R-package *mgcv*

```
> summary(fitRubber2)

Family: gaussian
Link function: identity

Formula:
loss ~ Hclass + s(tens, bs = "cs", k = 15)

Parametric coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)    272.22      14.66  18.569 2.79e-15 ***
Hclassmedium   -78.99      18.89  -4.181 0.000363 ***
Hclasshard    -169.55      19.36  -8.757 9.29e-09 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘
```

R-package *mgcv*

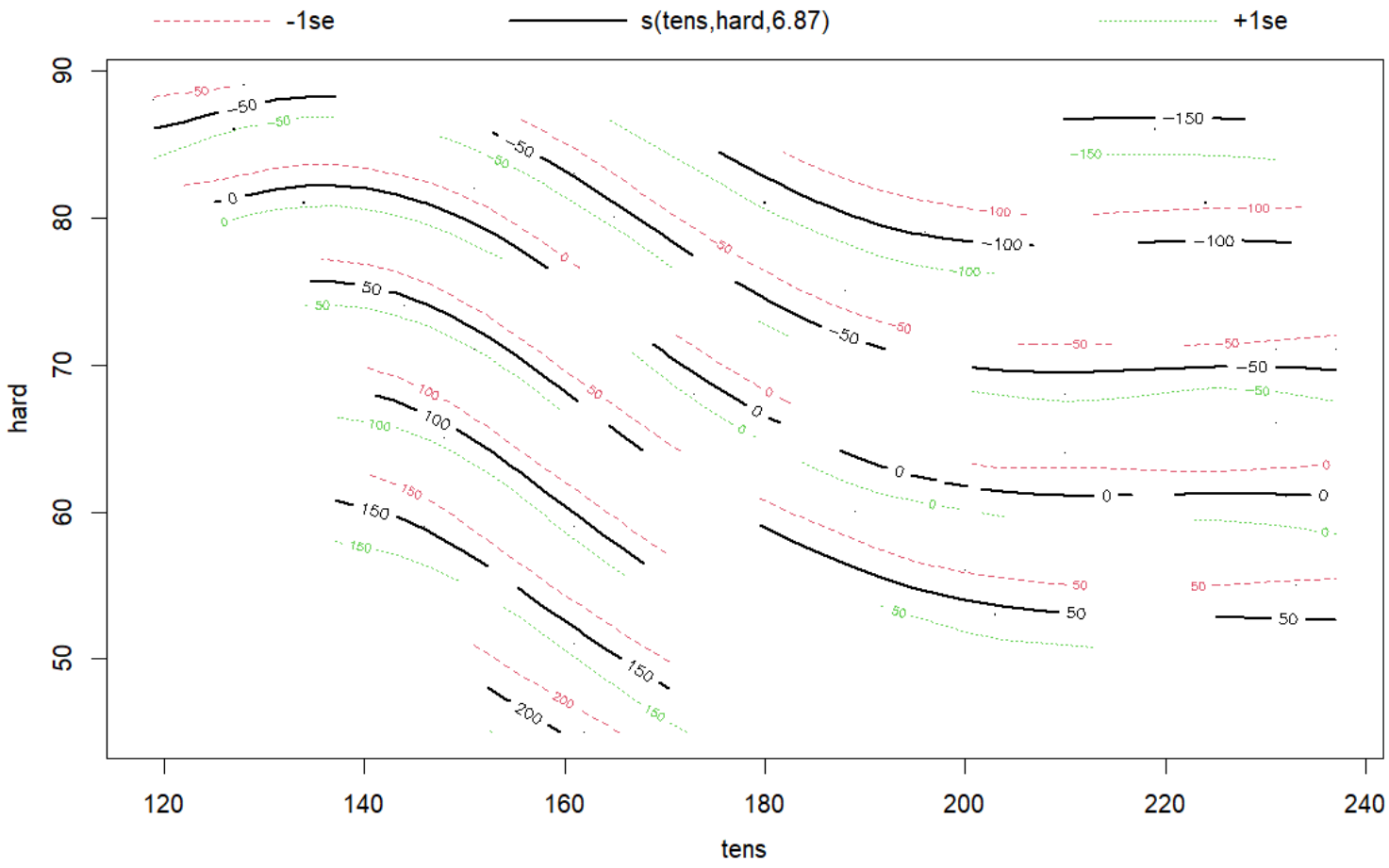
```
> summary(fitRubber2)
Parametric coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)    272.22      14.66  18.569 2.79e-15 ***
Hclassmedium   -78.99      18.89  -4.181 0.000363 ***
Hclasshard    -169.55      19.36  -8.757 9.29e-09 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘

Approximate significance of smooth terms:
          edf Ref.df     F  p-value
s(tens) 4.151     14 3.267 1.62e-05 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘

R-sq.(adj) =  0.823   Deviance explained =   86%
GCV = 1804.3  Scale est. = 1374.2    n = 30
```

R-package *mgcv*

```
> fitRubber3 <- gam(loss~s(tens, hard, bs="tp", k=15),
+ family=gaussian, data=Rubber)
> plot(fitRubber3, pages=1)
> summary(fitRubber3)
```

Exercise 7

R-package *mgcv*



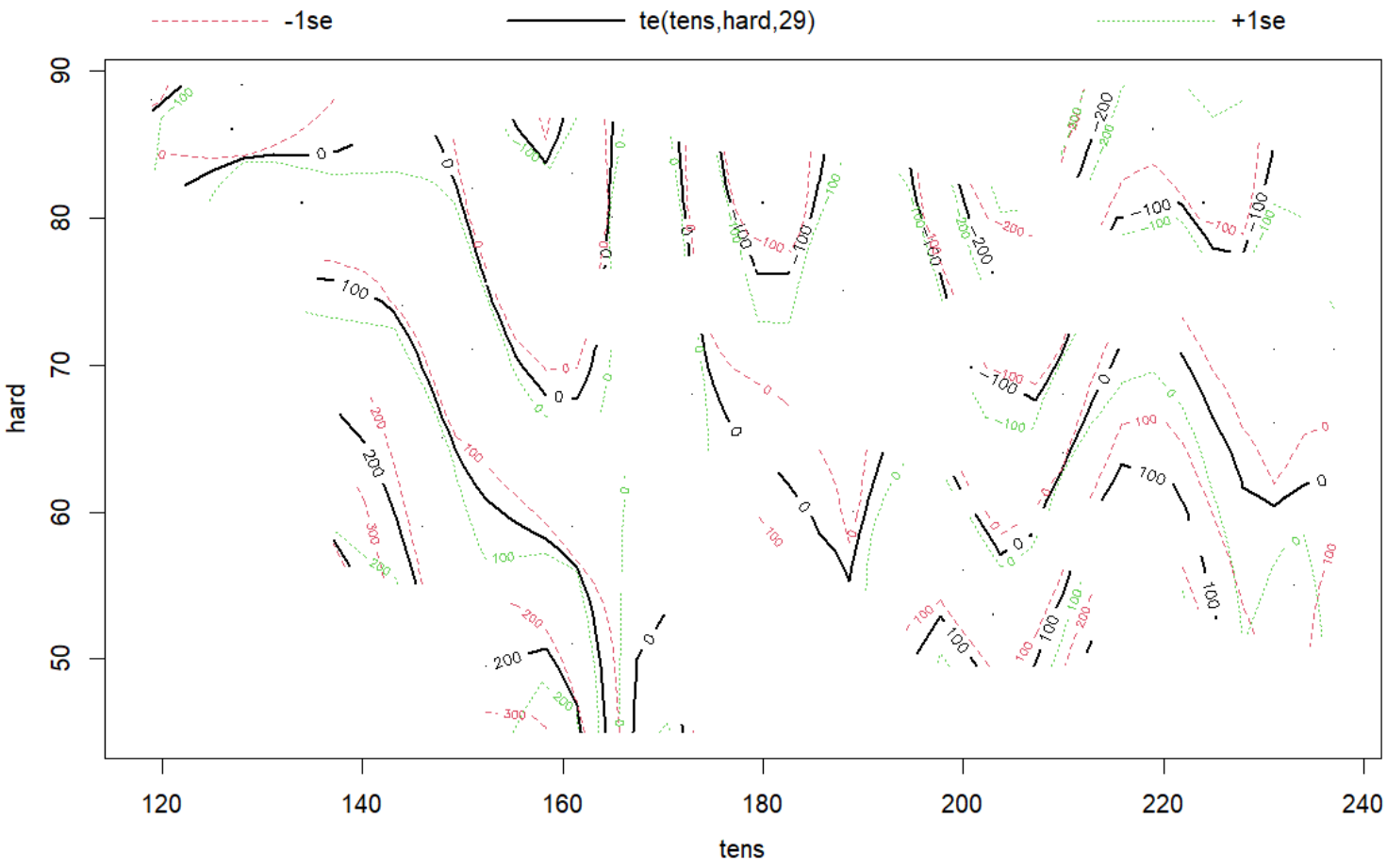
R-package *mgcv*

```
> fitRubber4 <- gam(loss~te(tens, hard, bs="cr", k=15),
+ family=gaussian, data=Rubber)
> plot(fitRubber4, pages=1)
> summary(fitRubber4)
```

R-package *mgcv*

```
Formula:
loss ~ s(tens, hard, bs = "tp", k = 15)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   175.43       4.88   35.95   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘

Approximate significance of smooth terms:
               edf Ref.df     F p-value
s(tens,hard) 6.867  8.959 31.96  <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘

R-sq.(adj) =  0.908   Deviance explained =   93%
GCV = 968.41  Scale est. = 714.45    n = 30
```

R-package *mgcv*



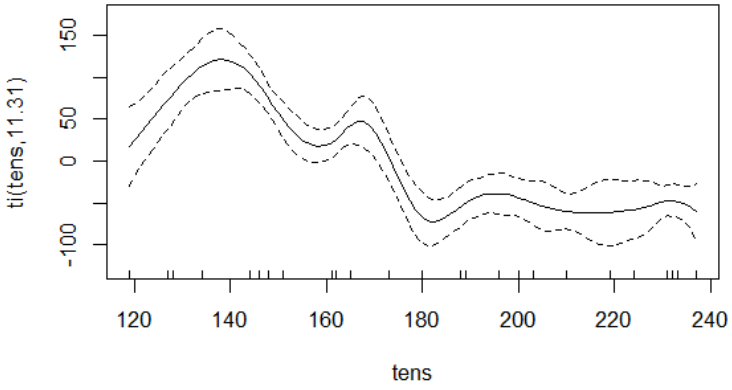
R-package *mgcv*

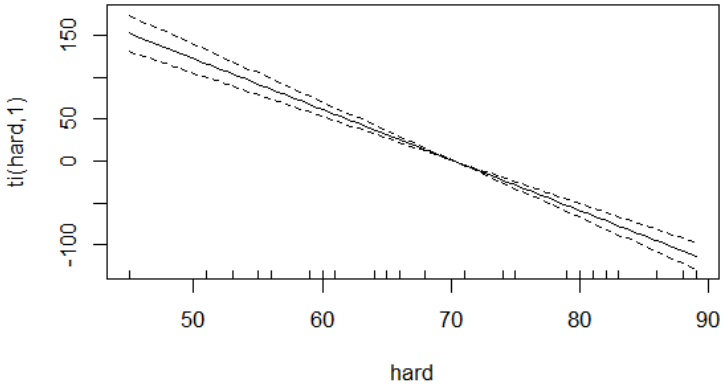
```
> fitRubber5 <- <- gam(loss~ti(tens, bs="cr", k=15)+
+ ti(hard, bs="cr", k=15)+ti(tens,hard,bs="cr"),
+ family=gaussian, data=Rubber)
> plot(fitRubber5, pages=1)
> summary(fitRubber5)
```

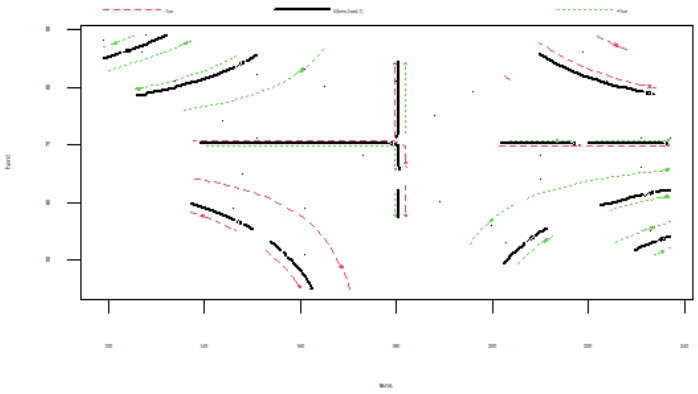
R-package *mgcv*

```
Formula:
loss ~ te(tens, hard, bs = "cr", k = 15)

Parametric coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 1.754e+02  7.326e-03   23946    0.998

Approximate significance of smooth terms:
              edf Ref.df       F p-value
te(tens,hard)  29     29 4818635 0.00036 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘

R-sq.(adj) =      1   Deviance explained =  100%
GCV =  379.2  Scale est. = 0.0016102  n = 30
```

R-package *mgcv*







R-package *mgcv*

```
Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  178.274      3.954   45.09   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘

Approximate significance of smooth terms:
               edf Ref.df     F p-value
ti(tens)      11.3  12.76  23.0  <2e-16 ***
ti(hard)       1.0   1.00 201.6  <2e-16 ***
ti(tens,hard)  1.0   1.00   1.7   0.211
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘

R-sq.(adj) =  0.958   Deviance explained = 97.7%
GCV = 624.37  Scale est. = 326.65    n = 30
```

Exercise 8